电子科技大学

计算机科学与工程学院

标 准 实 验 报 告

（实验）课程名称 计算机图形学

电子科技大学教务处制表

电 子 科 技 大 学

实　验　报　告

学生姓名：袁铭　学 号：2024080904027　　指导教师：肖逸飞

实验地点：主楼 A2-412　　实验时间：9-13 周周日

一、实验项目名称：计算机图形学——三维图形绘制与变换

二、实验学时： 16 学时

三、技术手段：

OpenGL 固定管线渲染、双缓冲机制、三维几何变换、观察变换；OBJ 模型解析、顶点法线计算、多光源光照系统、平面投影阴影、HUD 参数可视化；键盘回调、鼠标拖拽、窗口自适应。

四、实验内容及步骤：

（一）总体概况

本项目实现了一个功能完整的 OpenGL OBJ 模型查看器，核心目标是满足 “三维图形绘制、几何变换、观察变换” 三大实验要求。支持复杂 3D 模型加载、实时交互控制（模型的平移旋转和缩放、相机漫游控制）、光照阴影渲染及参数可视化。

（二）实现步骤

1. 实验环境配置与基础初始化

步骤 1：配置 OpenGL 开发环境，链接 opengl32.lib、glu32.lib、glut.lib 库文件，包含核心头文件；

步骤 2：通过 init()函数初始化 OpenGL 状态，启用深度测试（GL_DEPTH_TEST）、光照系统（GL_LIGHTING）、法线归一化（GL_NORMALIZE），设置背景色与材质属性；

步骤 3：验证环境有效性，确保窗口创建、双缓冲机制正常工作。



图 1 窗口创建、双缓冲机制正常工作

2. 3D 物体绘制（OBJ 模型加载）

功能说明：支持导入 OBJ 格式模型，自动解析顶点、面数据并预处理，满足实验 “加载复杂 3D 物体”；

实现步骤：

编写 loadOBJ()函数，读取 OBJ 文件中的 v（顶点）和 f（面）标签，处理负索引并将多边形自动三角化；

计算模型包围盒，对顶点进行居中平移和统一缩放（desiredMaxSize=2.0f），确保模型适配视口；

调用 computeNormals()函数，通过三角面叉乘计算面法线，累加后归一化得到顶点法线（为光照做准备）；

示例效果：成功加载 OBJ 模型。

核心代码关联：loadOBJ()解析模型数据，display()函数通过 glDrawElements()绘制三角面。

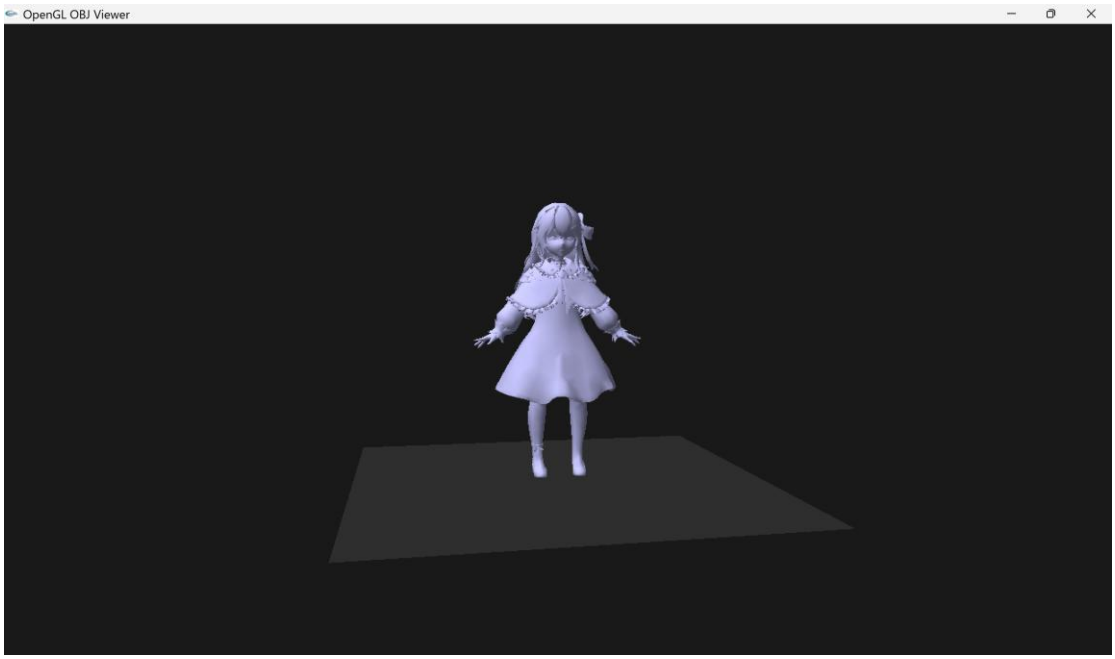


图 2 obj 模型正常导入显示

3. 几何变换（交互控制）

功能说明：支持模型的平移、缩放交互，满足实验 “设计键盘交互接口”要求；

实现步骤：

定义全局变量 g_modelPos（平移量）和 g_modelScale（缩放比例）；

编写 specialKeys()函数，绑定箭头键控制模型平移（X/Z 轴）、PageUp/PageDown 控制 Y 轴平移，步长随模型半径自适应（step = g_modelRadius * 0.05f）；

绑定键盘[/]键（keyboard()函数），控制模型缩小 / 放大，限制最小缩放比例（0.0001f）；

绑定 R 键重置模型变换（恢复到原点，缩放比例 1.0f）；

变换应用：在 display()中通过 glTranslatef()和 glScalef()修改模型矩阵，实现实时变换。

操作演示：按左箭头键→模型向左平移；按]键→模型放大 10%；按 R 键→模型复位。

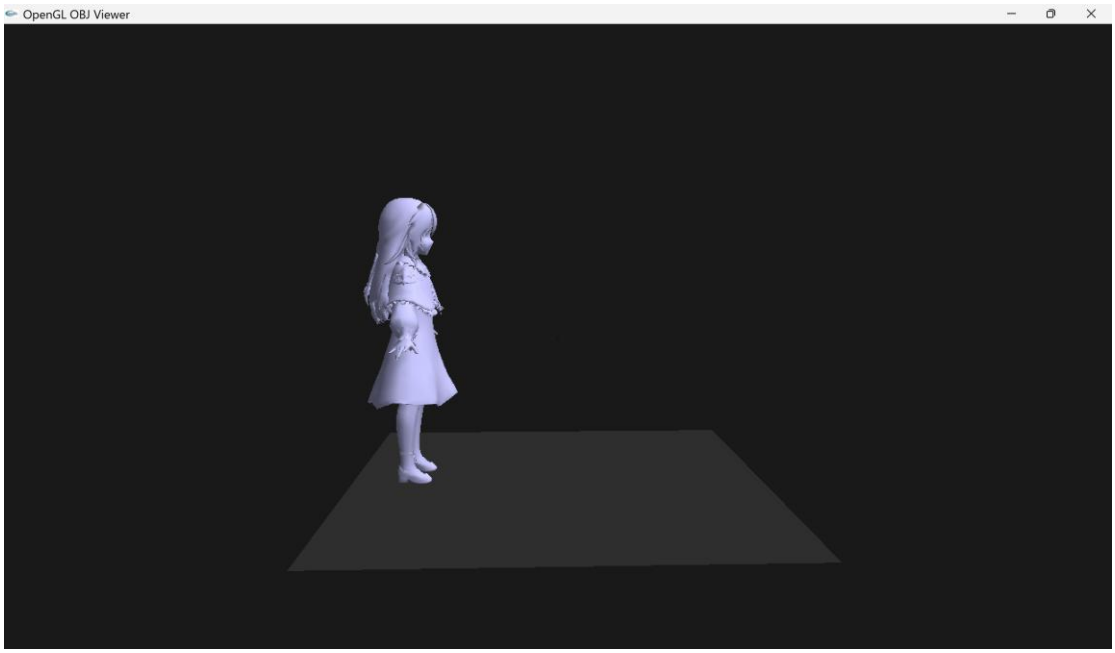


图 3 模型向左移动

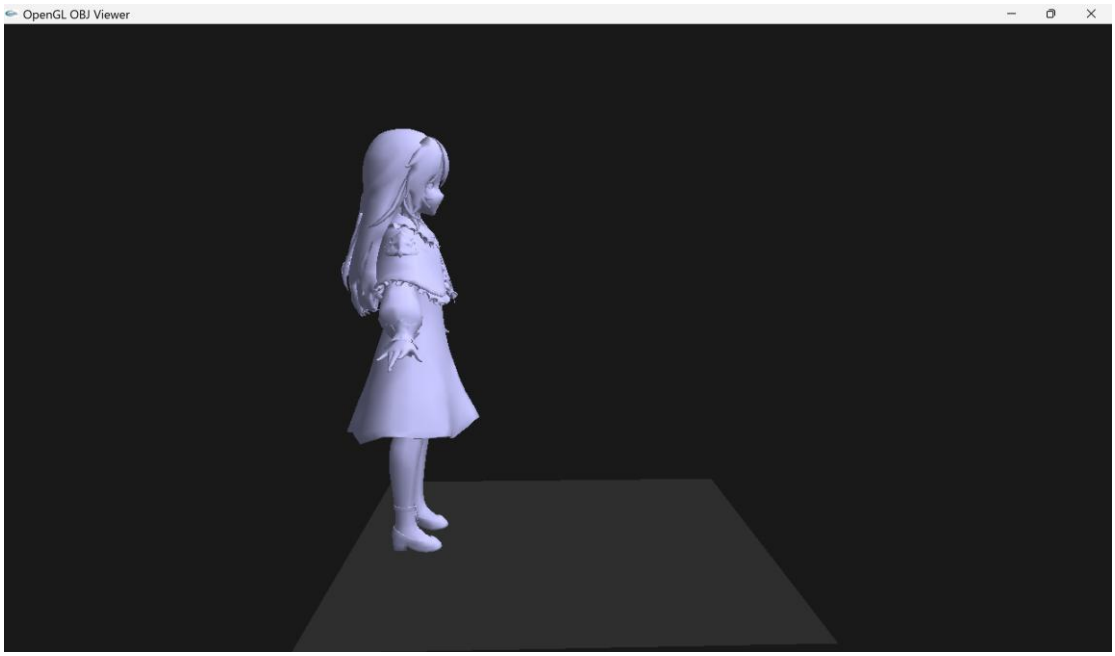


图 4 模型放大

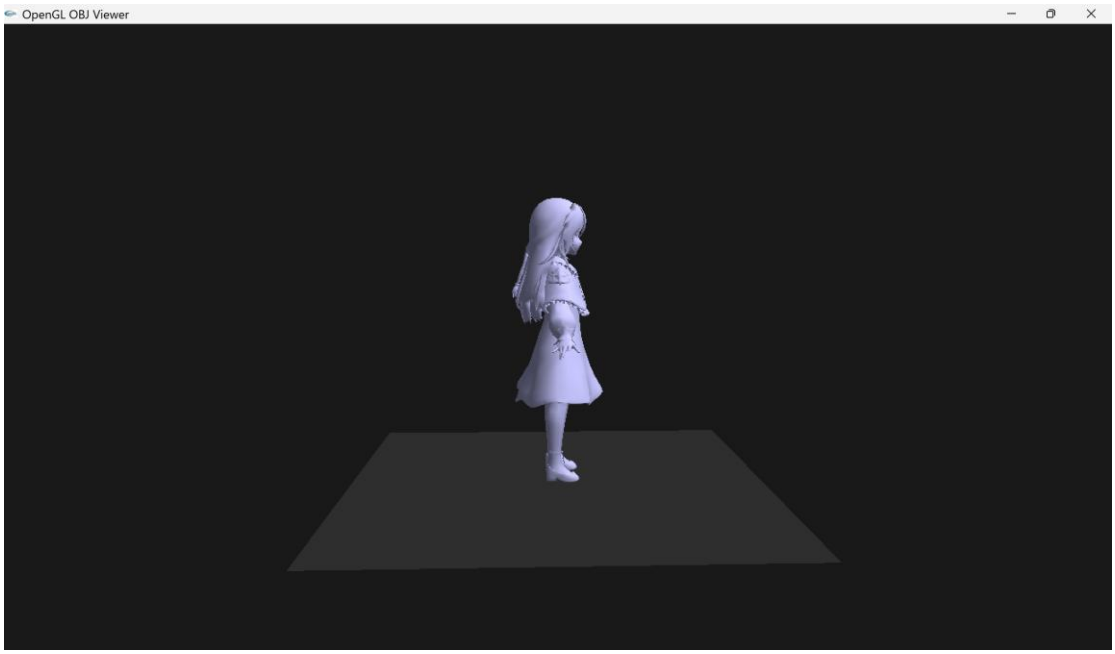


图 5 模型恢复原位

4. 观察变换（相机控制）

功能说明：支持相机移动、旋转切换，实现相机漫游，满足实验 “相机漫游” 要求；

实现步骤：

定义相机参数 g_eye（相机位置）、g_center（观察中心）、g_yaw（偏航角）、g_pitch（俯仰角）；

相机平移：绑定 W/S/A/D 键控制相机前后左右移动，Q/E 键控制上下移动（keyboard()函数），基于相机局部坐标系（forward/right 向量）计算移动方向；

相机旋转：通过 mouse()和 mouseMotion()函数，绑定鼠标左键拖拽，将鼠标偏移量转换为偏航角 / 俯仰角，再通过球面坐标公式更新 g_eye 位置，实现环绕观察；

示例效果：鼠标拖拽→相机环绕模型旋转；按 WSADQE 键→相机变换。

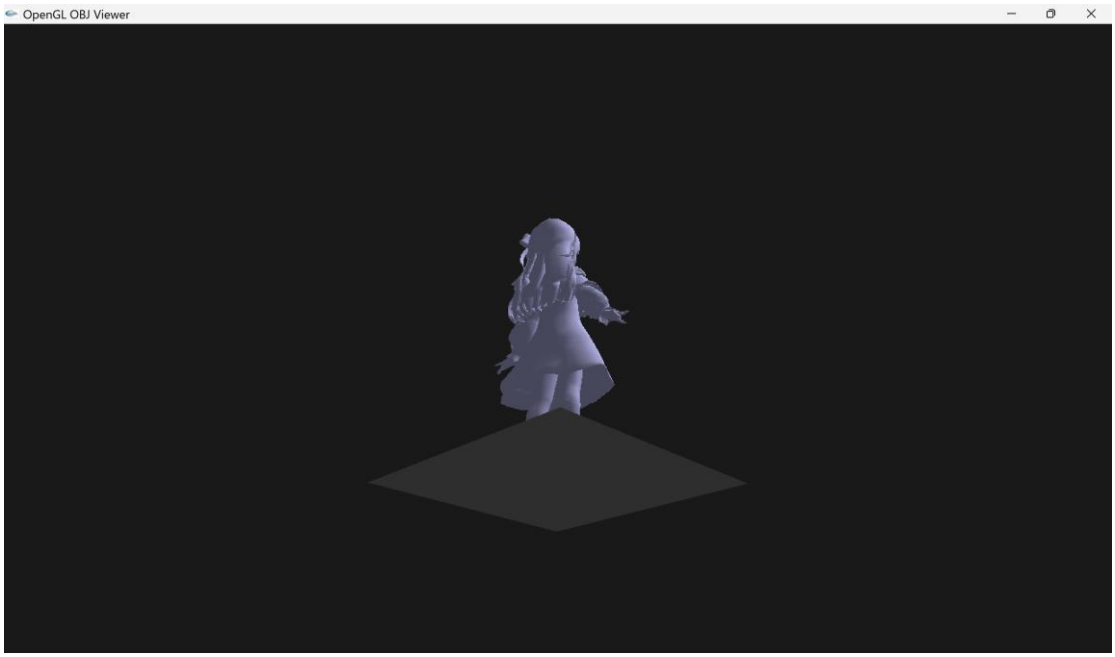


图 6 鼠标拖拽相机视角变换展示（1）

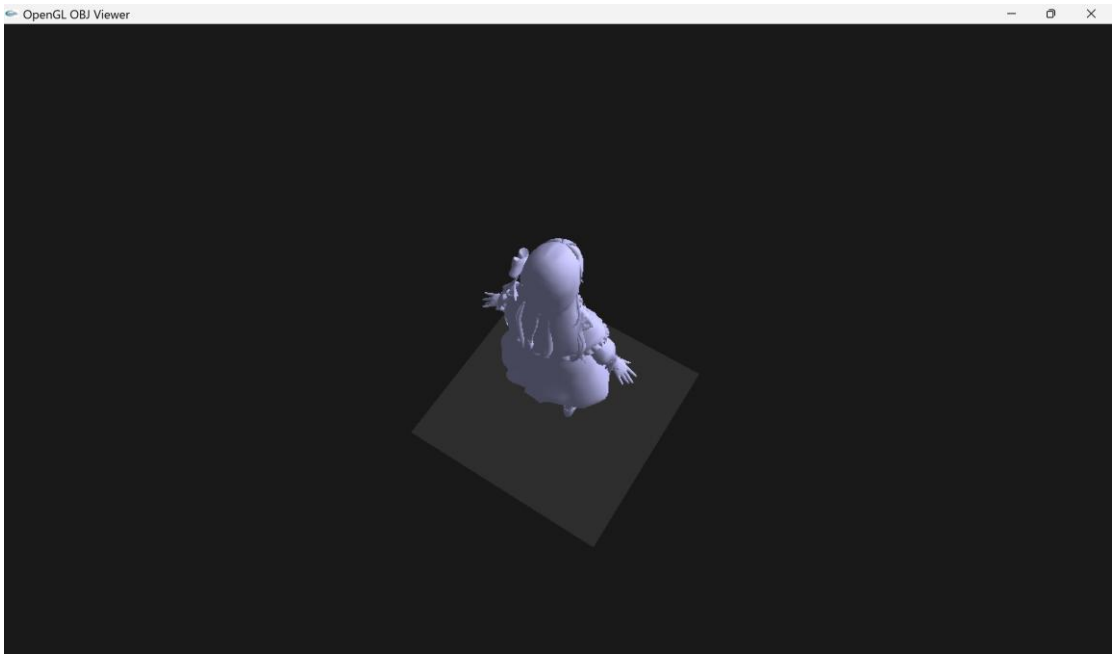


图 7 鼠标拖拽相机视角变换展示（2）

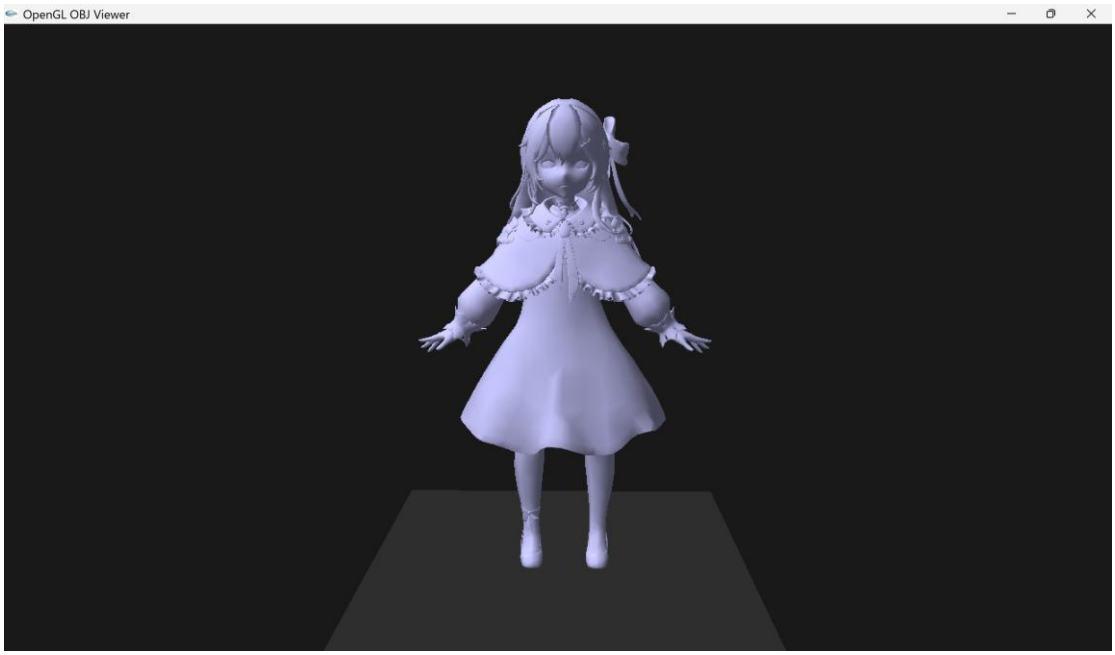


图 8 按 W 键相机靠近模型

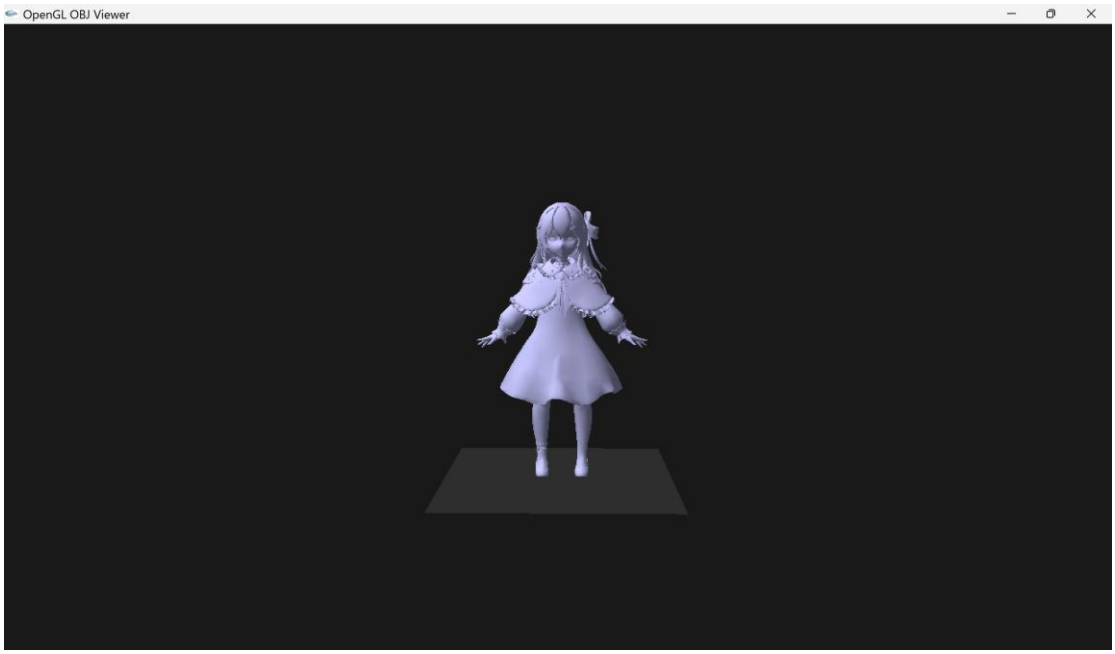


图 9 按 S 键相机远离模型

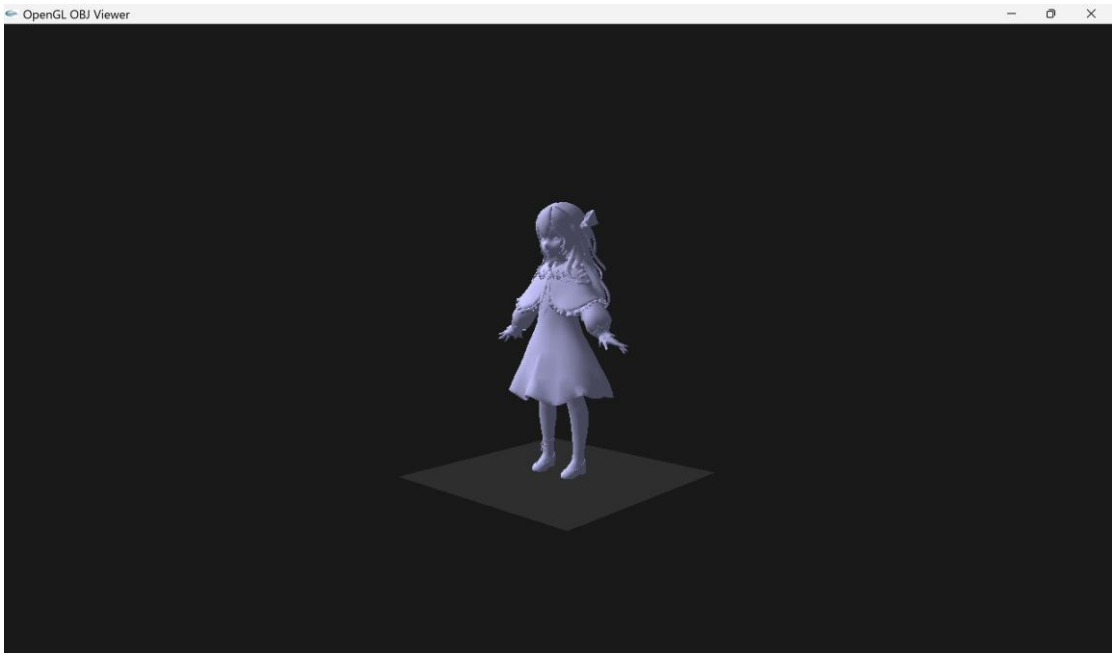


图 10 按 D 键相机向右旋转

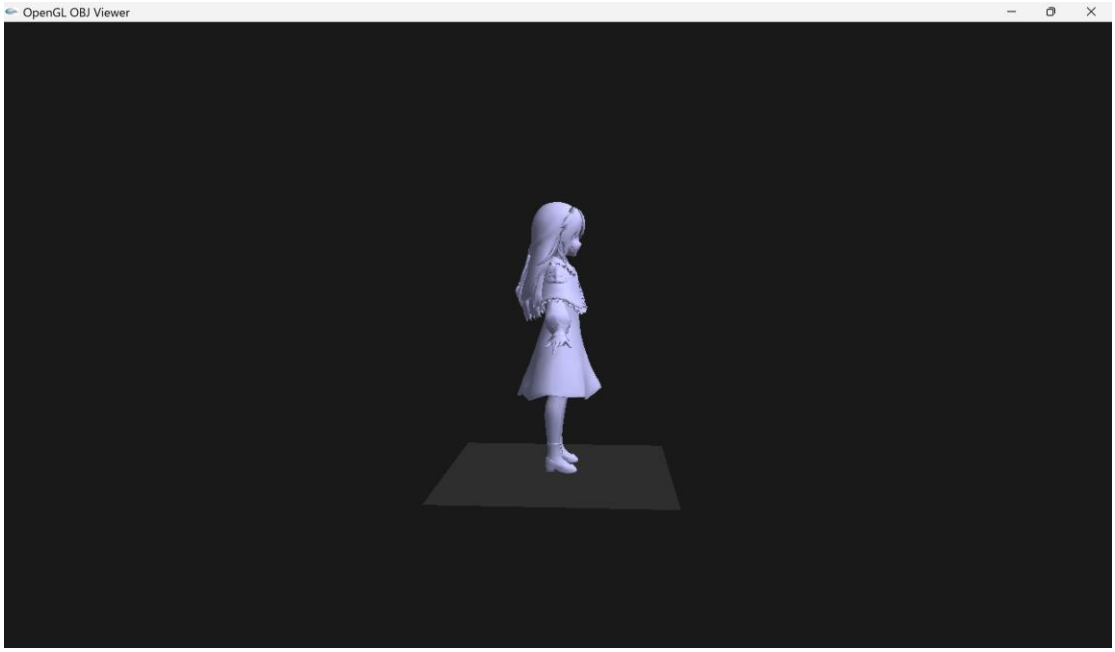


图 11 按 A 键相机向左旋转

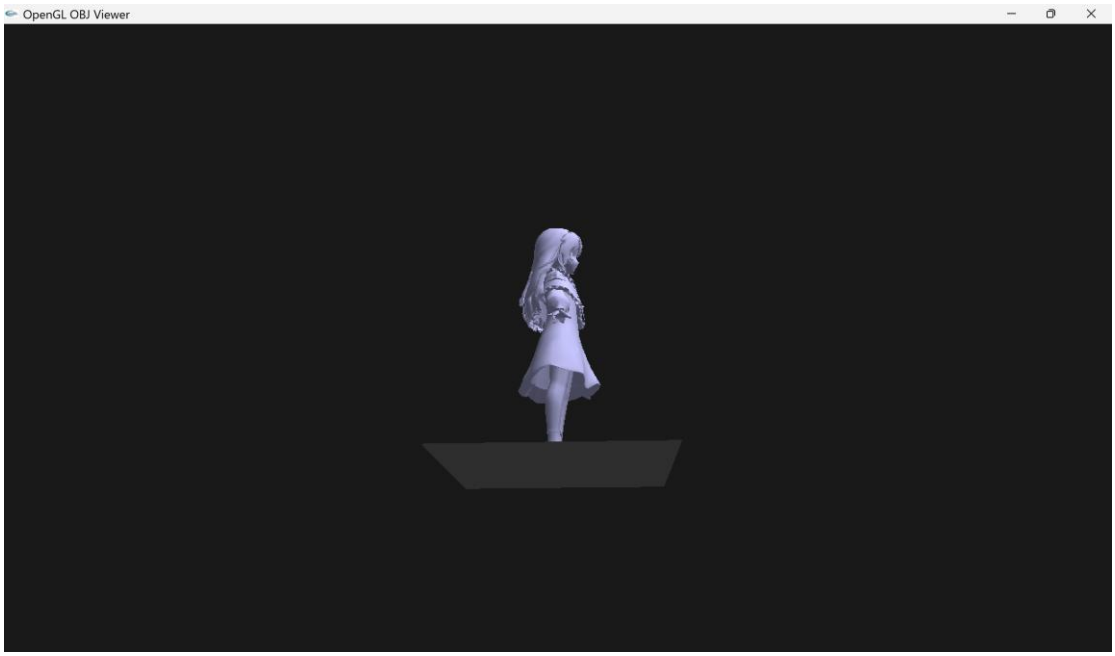


图 12 按 Q 键相机向上仰视

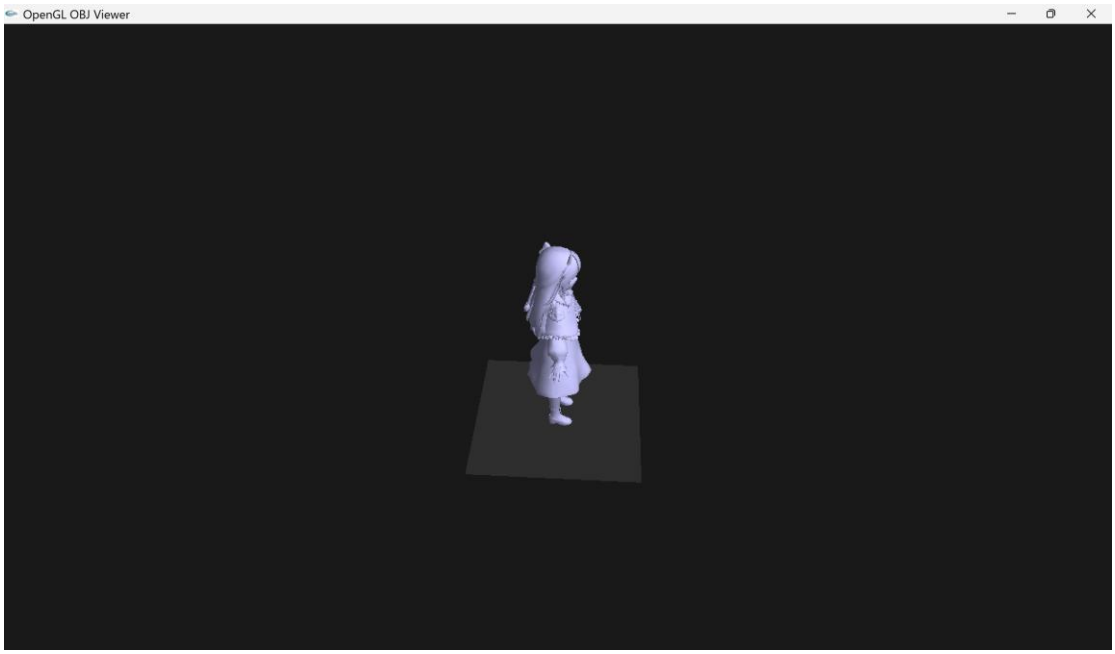


图 13 按 E 键相机向下俯视

5. 光照与阴影优化

功能说明：实现多光源光照和平面投影阴影，增强模型立体感；

实现步骤：

配置双光源系统：GL_LIGHT0 为主光（点光），GL_LIGHT1 为辅助光（支持方向光 / 点光切换），通过 updateLights()函数更新光照参数；

阴影实现：采用平面投影法，computeShadowMatrix()计算光源到地面的投影矩阵，drawShadowOnPlane()绘制半透明阴影，通过 GL_POLYGON_OFFSET_FILL 避免

深度冲突；

优化改进：缩小地面绘制范围，启用 GL_SCISSOR_TEST 裁剪多余阴影，解决阴影平面阻挡问题。



图 14 模型正常光影效果展示

6. HUD 参数可视化

功能说明：实时显示模型参数、渲染模式、交互提示，方便调试与操作；

实现步骤：

编写 drawText()函数，通过 glutBitmapCharacter()绘制位图文字；

drawHUD()函数整合顶点数、三角形数、相机位置、模型变换参数、光照状态、交互控件等信息，显示在屏幕左上角；

支持 H 键切换 HUD 显示 / 隐藏。

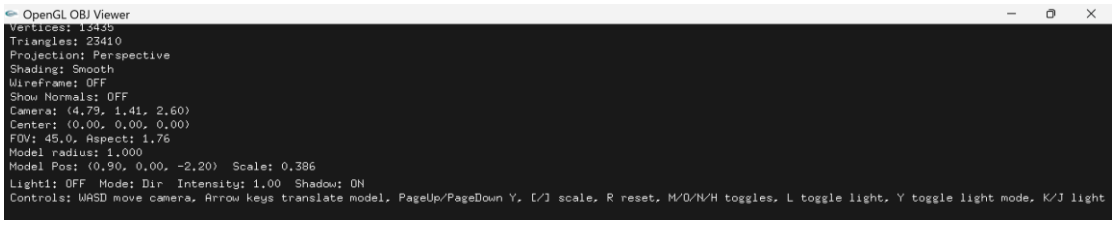


图 15 HUD 参数可视化展示

（三）最终结果展示

模型渲染：项目成功加载 OBJ 模型展示，支持键盘选择填充/线框模式切换，法线可视化（N 键）；

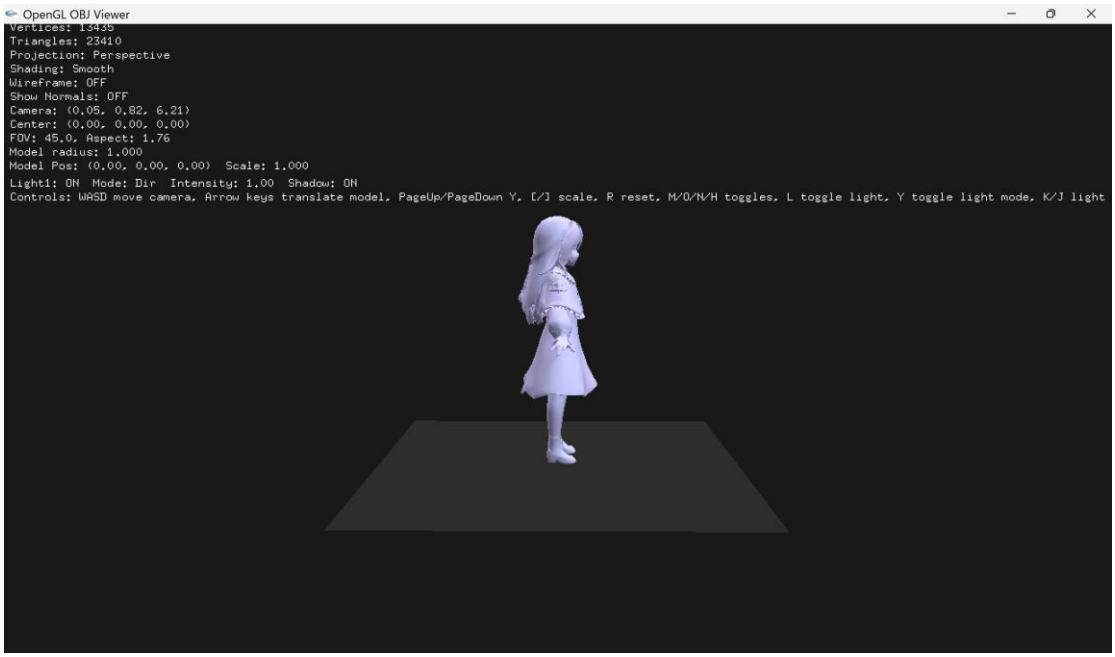


图 16 模型正常加载界面

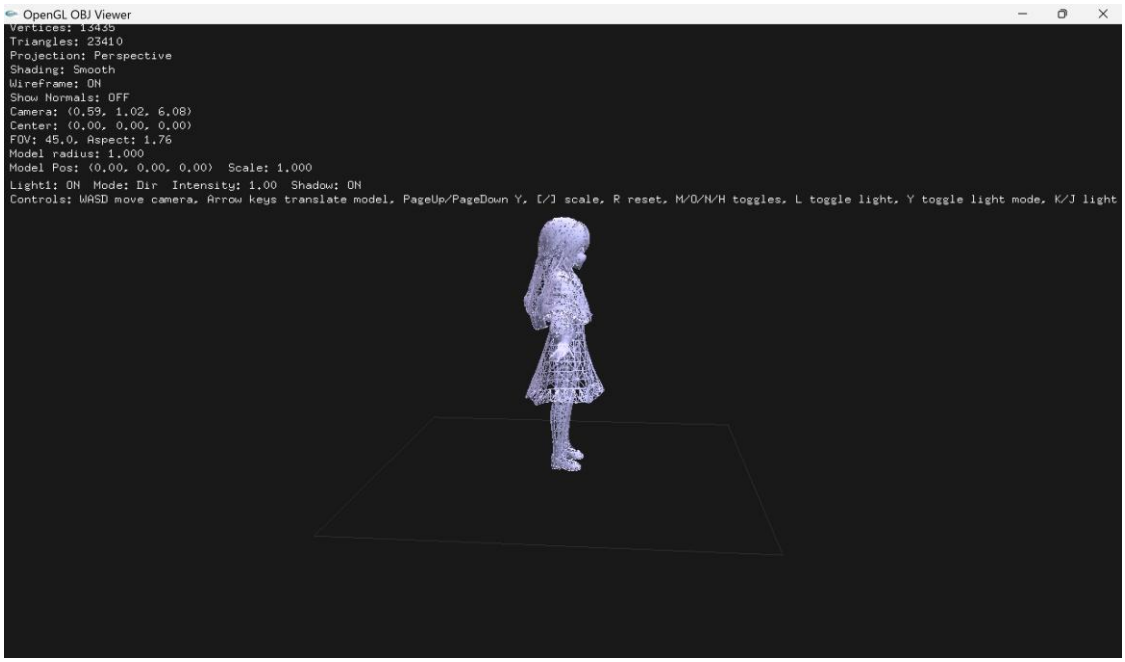


图 17 模型线框模式展示

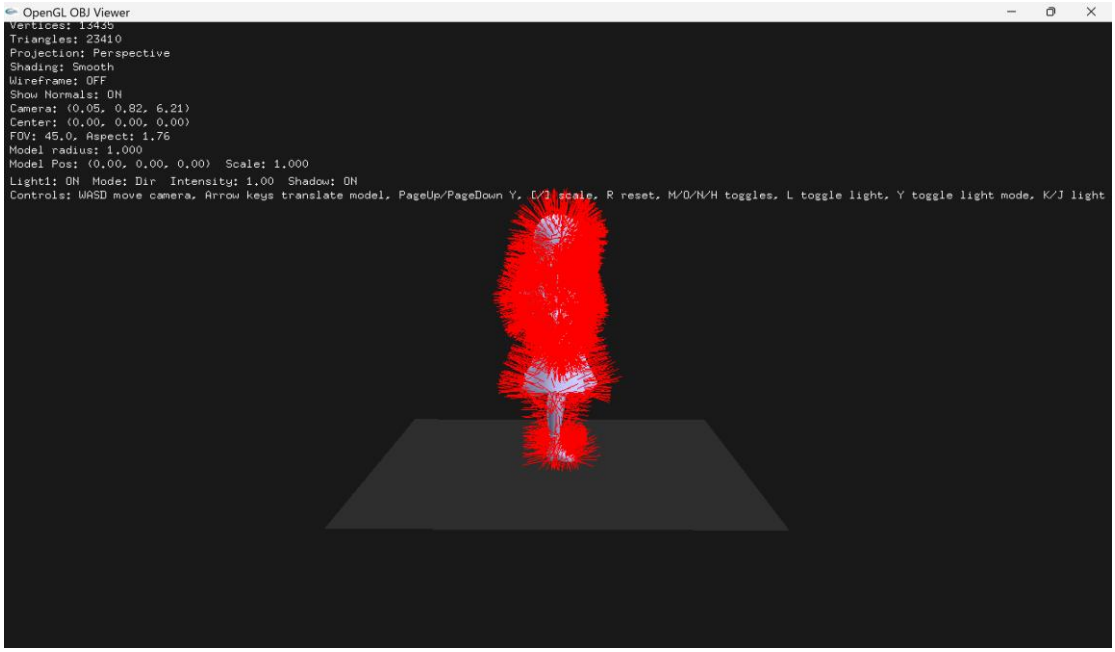


图 18 模型法线可视化

交互控制：模型平移/缩放、相机移动/旋转响应实时，无延迟；

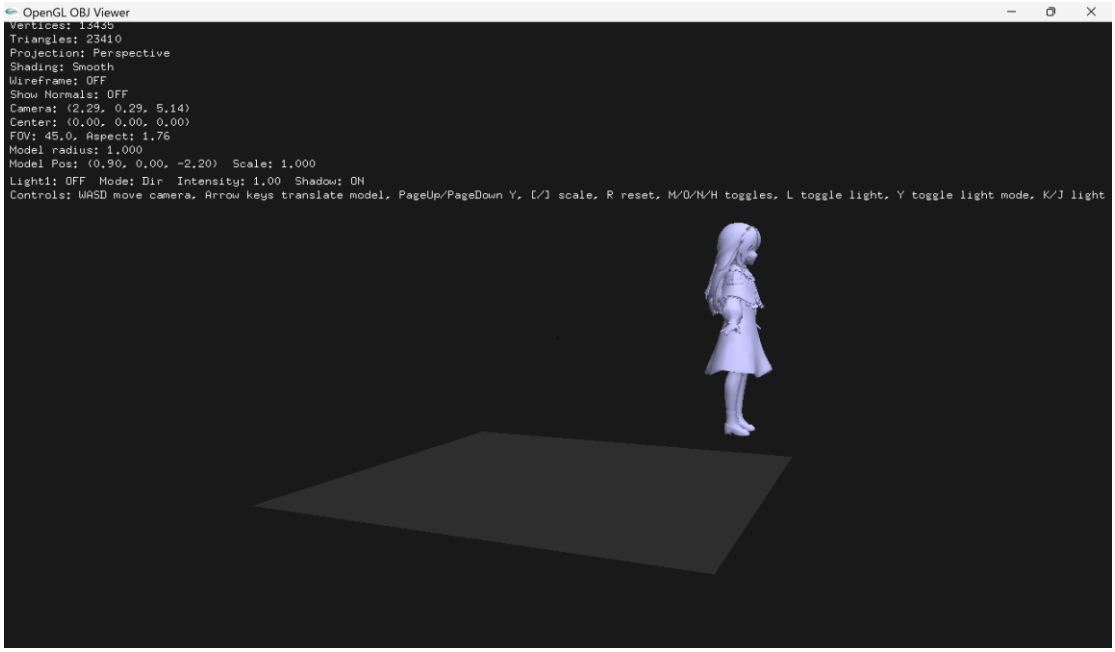


图 19 模型平移



图 20 相机旋转

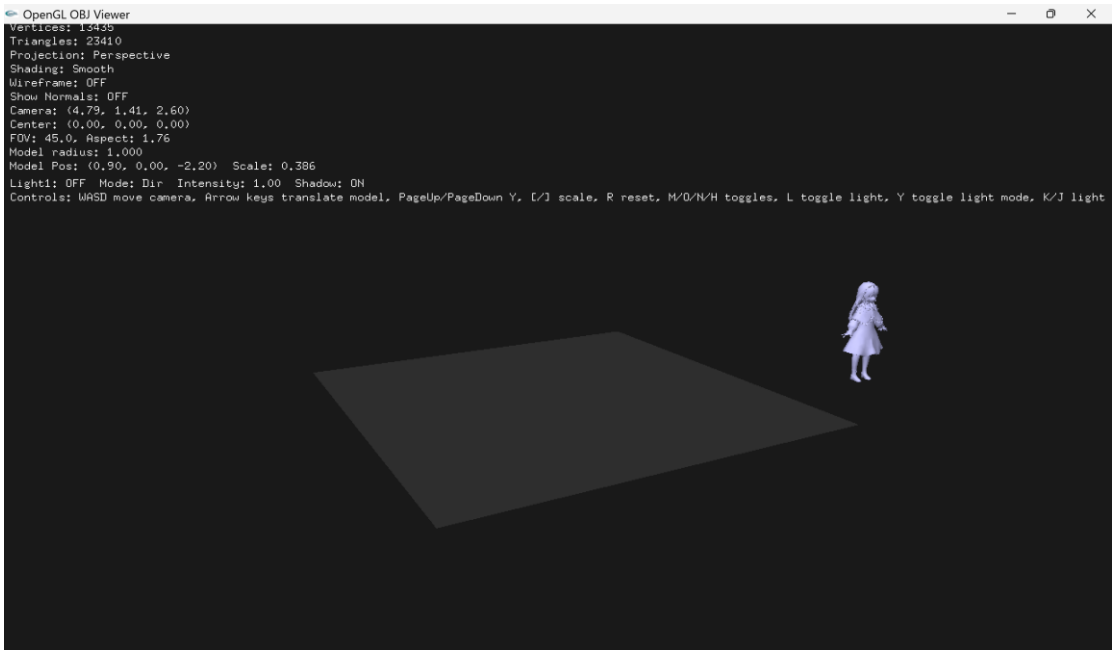


图 21 模型缩放

光照阴影：模型表面明暗过渡自然，阴影贴合地面，无多余阻挡平面；

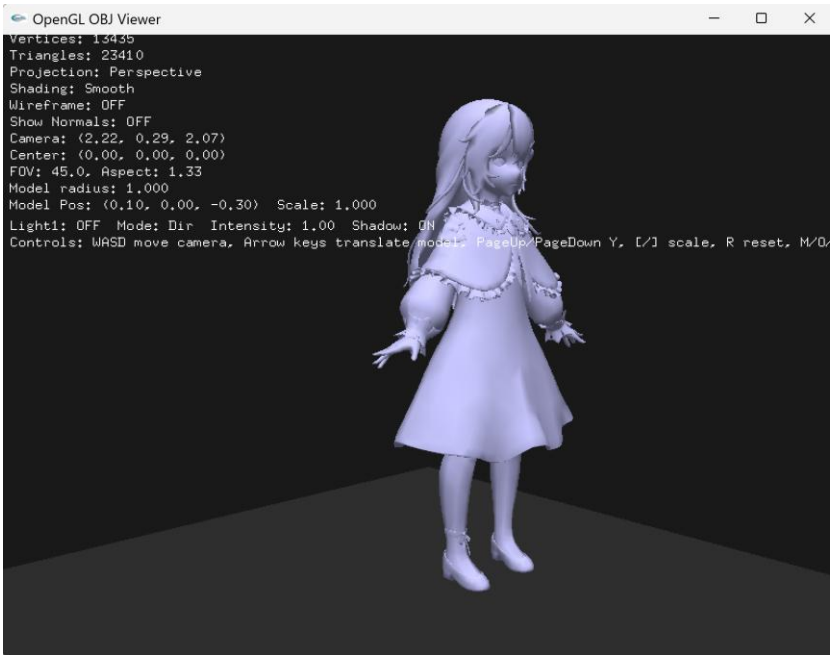


图 22 模型正常光影效果展示

稳定性：窗口缩放时图形不变形，程序支持正常退出（ESC 键）。

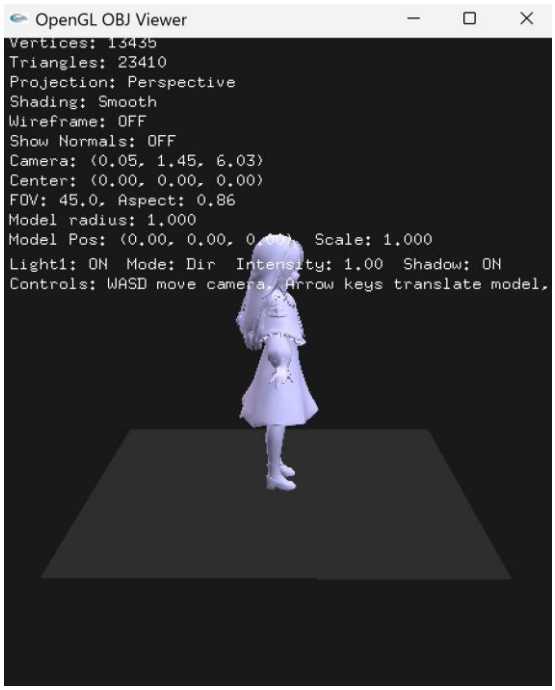


图 23 窗口缩放图形不变形变形



图 24 程序正常退出

五、总结及心得体会：

（一）遇到的问题及解决办法

1.OBJ 模型加载失败：问题原因是文件路径不合规，解决办法是修改路径用正斜杠替代反斜杠；

2.阴影平面阻挡：问题原因是地面高度计算忽略模型变换、深度偏移参数错误，解决办法是同步模型缩放 / 平移计算地面高度，调整 glPolygonOffset(1.0f, 1.0f)，启用剪裁测试；

3.相机旋转倒立：问题原因是俯仰角未限制范围，解决办法是将 g_pitch 限制在 [-89°，89°]，避免相机翻转。

（二）心得体会

1.深入理解了 OpenGL 渲染管线：从顶点数据→矩阵变换→光照计算→像素渲染的完整流程，体会到 “矩阵叠加” 在变换中的核心作用；

2.细节优化决定效果：阴影的深度偏移、地面范围的限制、法线的归一化等细节，直接影响渲染质量，需耐心调试；

3.模块化编程的优势：将模型加载、交互控制、光照阴影拆分为独立函数，便于维护与功能扩展。

六、对本实验过程及方法、手段的改进建议：

（一）对实验教学的建议

1.建议增加 OpenGL 核心管线（VAO/VBO）的教学内容，适应现代图形学开发趋势；

2.提供更多 OBJ 模型素材及解析示例，帮助学生解决模型加载中的常见问题；

3.增设小组讨论环节，分享交互设计与效果优化的经验。

（二）对项目的改进想法

1.功能扩展：增加模型旋转功能（绑定键盘快捷键或鼠标滚轮），同时支持纹理贴图，解析 OBJ 文件中的 vt 标签和材质文件（mtl）。

2. 性能优化：改用顶点缓冲对象（VBO）存储顶点数据，替代固定管线的 glVertexPointer，提升绘制效率，优化阴影计算，减少重复矩阵运算；

3.用户体验：增加快捷键提示弹窗，降低操作学习成本，支持模型文件选择对话框，无需手动修改代码路径，增加背景色切换功能，适配不同模型风格。

报告评分：

指导教师签字：

电子科技大学

计算机科学与工程学院

标 准 实 验 报 告

（实验）课程名称 计算机图形学

电子科技大学教务处制表

电 子 科 技 大 学

实　验　报　告

学生姓名：袁铭　学 号：2024080904027　　指导教师：肖逸飞

实验地点：主楼 A2 412　　实验时间：9-13 周周日

一、实验项目名称：计算机图形学——贝塞尔曲线与分形图形

二、实验学时： 16 学时

三、技术手段：

1. 双缓冲技术；2. 正交投影（gluOrtho2D）；3. 伯恩斯坦基函数；4. 复数迭代算法；5. HSV 颜色映射；6. 事件驱动交互（键盘/鼠标/定时器）；7. 多视口划分（glViewport）。

四、实验内容及步骤：

（一）总体概况

本实验完成 “贝塞尔曲线” 与 “Mandelbrot 分形图形” 两大模块开发，完全覆盖实验要求：

贝塞尔曲线模块：实现单 / 多条曲线生成、控制点显示与交互、自动动画；

Mandelbrot 分形模块：实现迭代次数输入、分形绘制、动画效果、多视口对比；

所有功能通过 OpenGL 渲染，支持实时交互与状态反馈。

（二）贝塞尔曲线模块实现

步骤 1：程序初始化与初始界面

代码逻辑：通过`init()`函数完成基础配置——初始化白色画布背景（`glClearColor(1.0f,1.0f,1.0f,1.0f)`）、设置正交投影（`gluOrtho2D(0, WINDOW_WIDTH, 0, WINDOW_HEIGHT)`），创建默认空曲线（`curves.emplace_back()`),并初始化随机数种子(`srand(time(nullptr))`,为后续动画做准备)。

界面效果：启动程序后，显示 800×600 白色画布，左上角状态栏提示“Current Curve: 1/1 Animation: Off”，左下角显示操作提示（左键添加/选中控制点、右键新建曲线等），无任何曲线或控制点（图 1）。



图 1-1 贝塞尔曲线程序初始界面

步骤 2：基础功能——添加控制点生成单条曲线

代码逻辑：

1. 左键点击画布时，`mouseFunc`判断点击位置：若未选中现有控制点，则调用`points.emplace_back(ox, oy)`添加新控制点（`ox`/`oy`为鼠标坐标转换后的逻辑坐标）；

2. `drawBezierCurves()`函数中，通过`calculateBezierPoint`（基于伯恩斯坦基函数+二项式系数）计算曲线上的点，用`GL_LINE_STRIP`绘制连续线段（循环 200 次确保曲线平滑），同时用红色`GL_POINTS`显示控制点，灰色线段连接控制点。

操作与效果：左键点击画布 3 次，添加 3 个控制点（生成 2 阶贝塞尔曲线），界面显示彩色平滑曲线、红色控制点及灰色连接线，曲线起点标注“#1”（曲线编号）（图 2）。

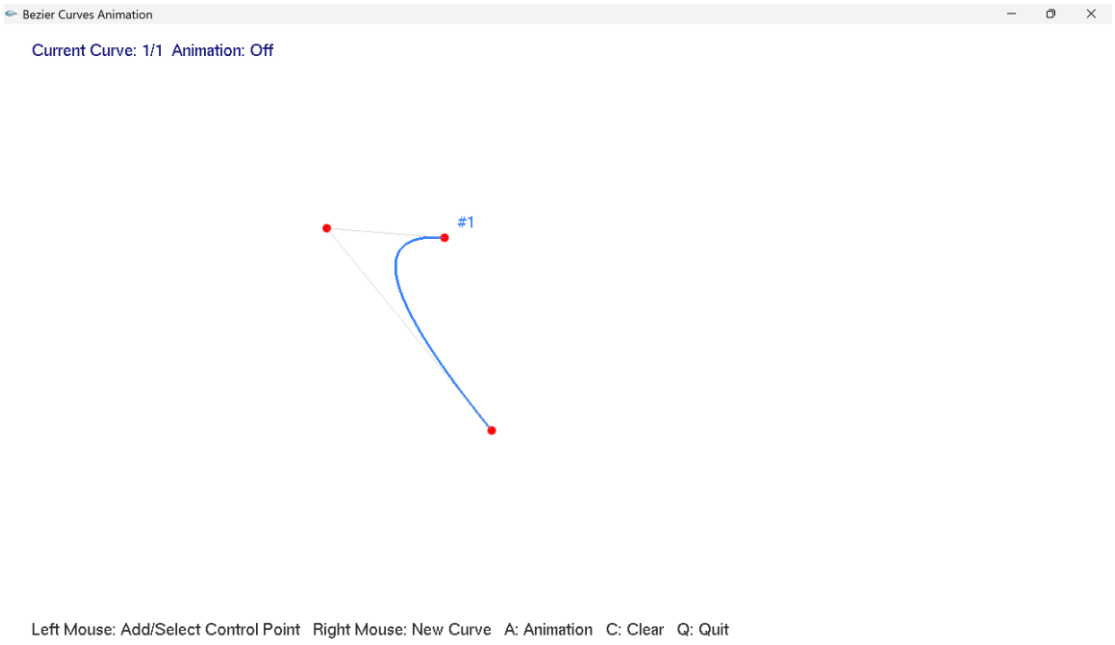


图 1-2 单条贝塞尔曲线绘制效果（3 个控制点）

步骤 3：创建并管理多条曲线

代码逻辑：右键点击画布时，`mouseFunc`调用`curves.emplace_back()`创建新曲线，更新`currentCurve`为新曲线索引（`curves.size()-1`），实现多条曲线独立管理；绘制时循环遍历`curves`向量，每条曲线用不同色区分。

操作与效果：右键点击画布，状态栏更新为“Current Curve: 2/2”，再左键点击 3 次添加新控制点，生成第二条贝塞尔曲线（颜色与第一条不同），两条曲线并行显示，各自标注“#1”“#2”编号（图 3）。

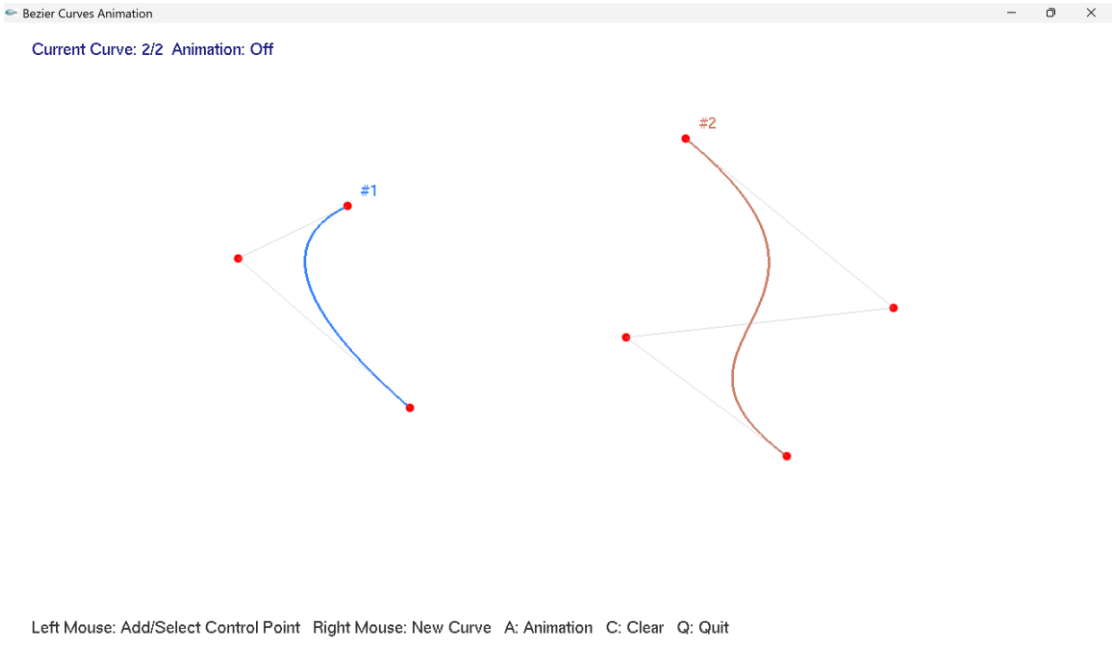


图 1-3 多条贝塞尔曲线并行显示效果

（注：截图需居中，清晰展示两条不同颜色的曲线、各自控制点及状态栏曲线数量更新）

步骤 4：鼠标拖拽控制点实时更新曲线

代码逻辑：

1. 左键点击控制点时，`mouseFunc`计算鼠标与控制点的欧氏距离（`<10.0f`判定为选中），设置`selectedPoint`为选中点索引，选中点用`glPointSize(16.0f)`黑色点+红色中心点标识；

2. 拖拽鼠标时，`motionFunc`实时更新`curves[currentCurve][selectedPoint]`的坐标，调用`glutPostRedisplay()`触发重绘，曲线随控制点位置即时变形。

操作与效果：鼠标移动至某红色控制点，按住左键拖拽，曲线实时跟随控制点变形（如将中间控制点上移，曲线向上凸起），其他控制点位置不变（图 4）。

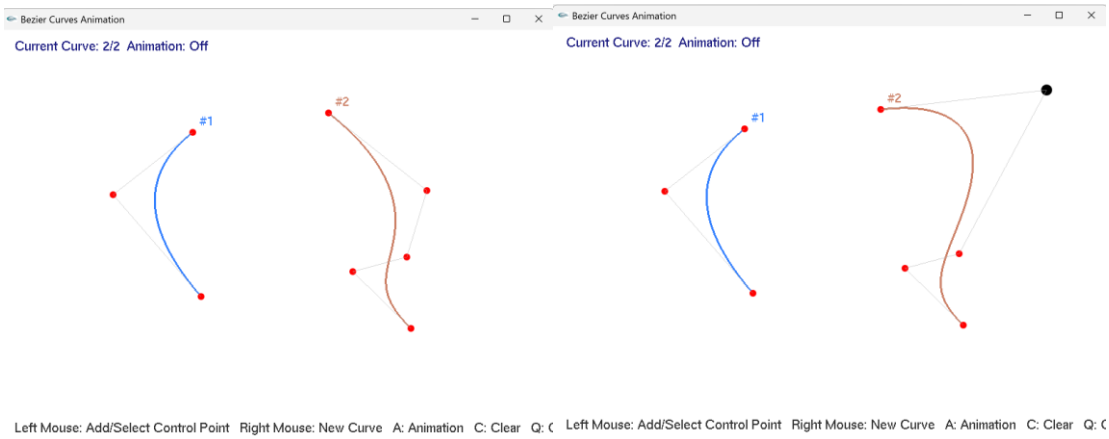


图 1-4 鼠标拖拽前后控制点实时更新曲线效果（左图为拖拽前）

步骤 5：开启控制点自动动画

代码逻辑：按“A”键时，`keyboardFunc`切换`isAnimation`状态；开启后，`animationFunc`每 50ms 执行一次——随机调整所有曲线控制点的坐标，并限制控制点在窗口内（`std::max/min`确保不超出边界），实时重绘曲线。

操作与效果：按“A”键，状态栏更新为“Animation: On”，所有控制点开始缓慢随机移动，曲线随之动态变形（如波浪状起伏），动画过程无闪烁（双缓冲保障）（图 5）。

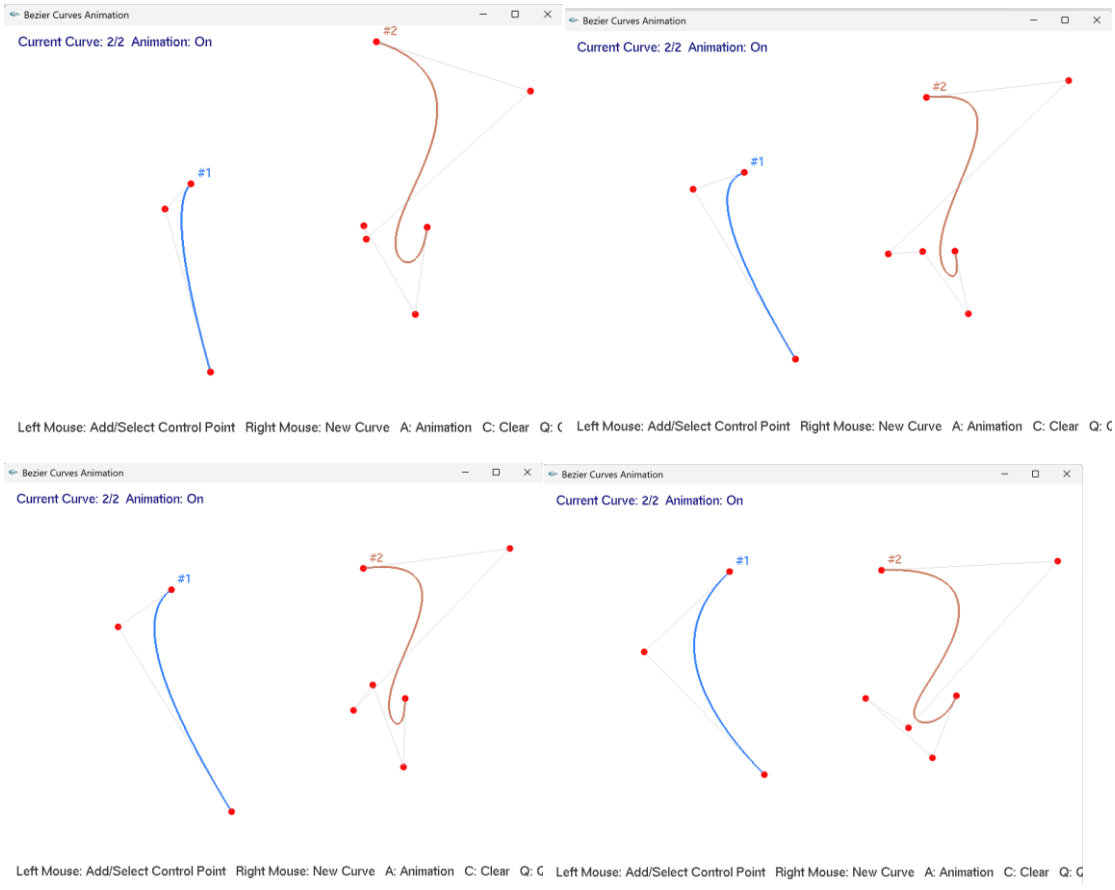


图 1-5 贝塞尔曲线自动动画效果（某四帧）

贝塞尔曲线模块最终结果展示及说明

最终界面：界面同时显示 3 条动态变形的贝塞尔曲线，包含黑色加粗描边的选中控制点、灰色控制点连线、曲线编号标签，状态栏实时显示“Current Curve: 3/3 Animation: On”，左下角保留操作提示（图 6）。

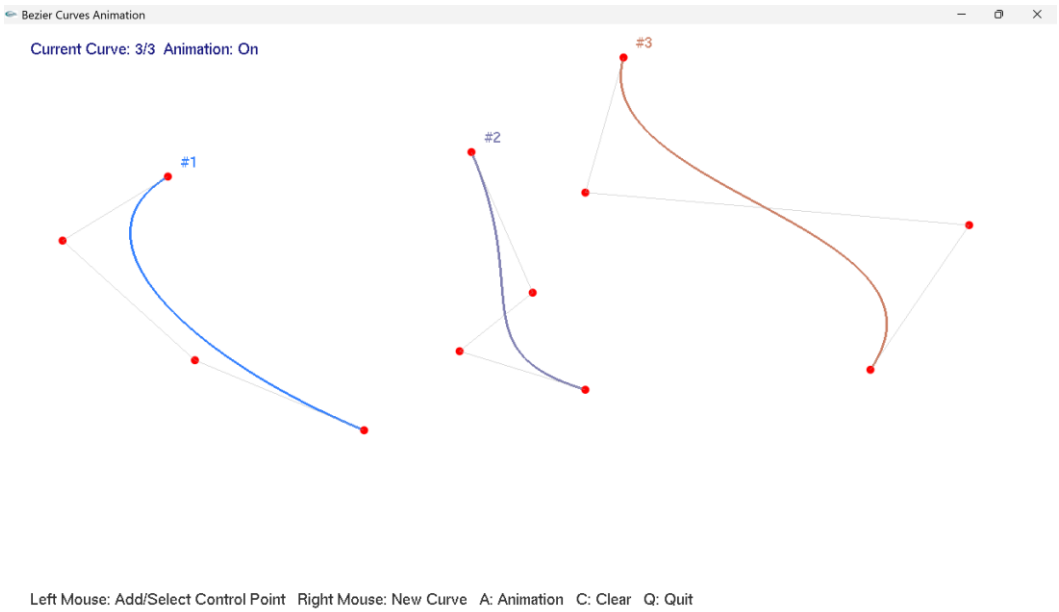


图 1-6 贝塞尔曲线模块最终综合效果

功能覆盖说明：完全实现实验要求——单条曲线生成+控制点显示、多条曲线、鼠标交互更新、自动动画，代码中`calculateBezierPoint`确保曲线平滑，`mouseFunc`/`motionFunc`保障交互流畅，`animationFunc`实现稳定动画，符合 OpenGL 1.0+GLUT 的技术要求。

（三）Mandelbrot 分形图形模块实现

步骤 1：程序初始化与初始界面

代码逻辑：init()函数配置黑色画布背景，通过 gluOrtho2D 建立正交投影；初始化全局参数（默认迭代次数 50、复平面范围 xMin=-2.0~xMax=1.0/yMin=-1.5~yMax=1.5，关闭动画、多视口及性能模式，渲染缩放 1（全分辨率））。

界面效果：启动后显示 900×900 黑色画布，渲染 50 次迭代的 Mandelbrot 集（黑色未逃逸点，彩色逃逸点）；底部状态栏显示 “Iter: 50 | Anim: OFF | View: Single | Scale:1/1”，左上角帮助面板展示操作说明，窗口标题为“Mandelbrot Set - Iter: 50 Scale:1/1”（图 1）。

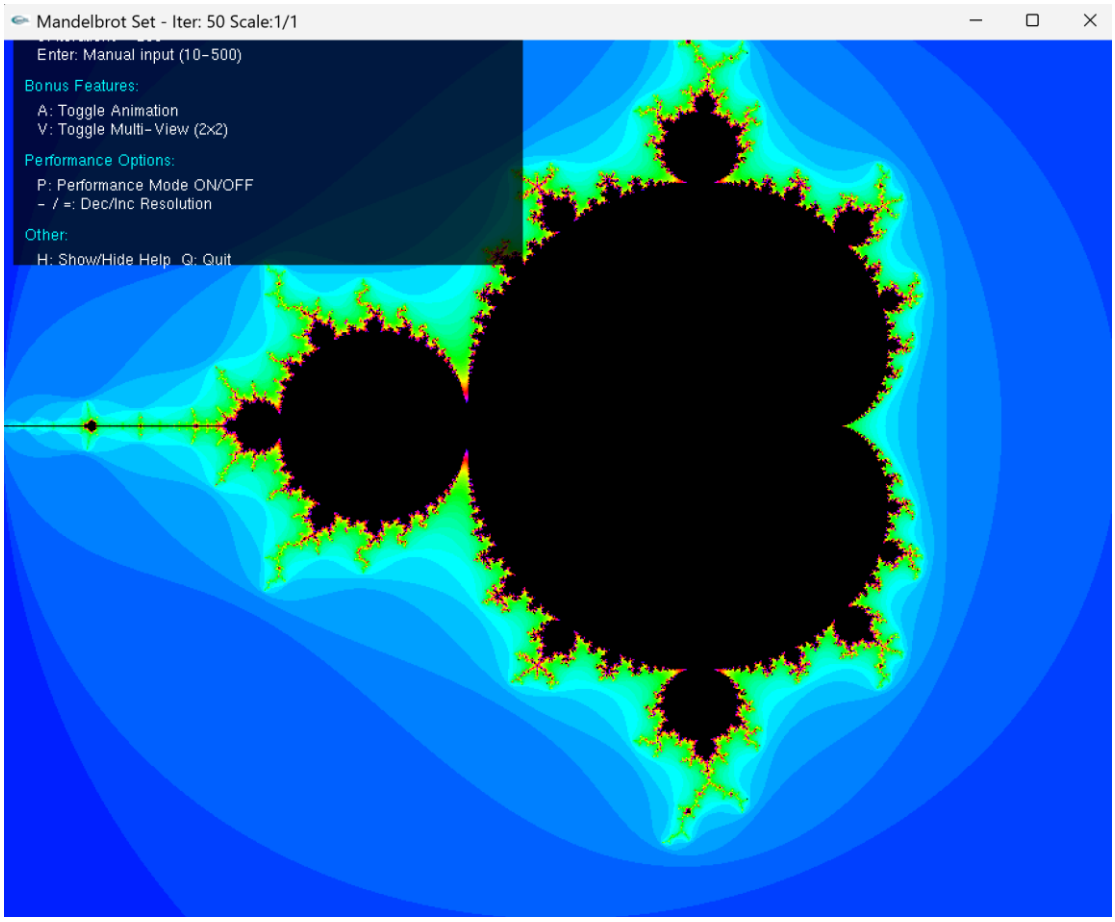


图 2-1 分形模块初始界面（50 次迭代，全分辨率）

步骤 2：基础功能 —— 迭代次数控制与性能优化绘制

代码逻辑：

迭代次数：支持 “1-5” 键快速设置（30/50/100/150/200）、Enter 键手动输入（范围 10-500）；

性能优化：mandelbrotIteration 采用实部分离计算和早期终止减少开销；drawSingleMandelbrot 通过 renderScale 实现降采样渲染（全分辨率逐点绘，半 / 四分之一分辨率绘矩形块）。

操作与效果：

按 “3” 键设迭代次数为 100，分形细节显著增强，状态栏更新为 “Iter：100”（图 2）；

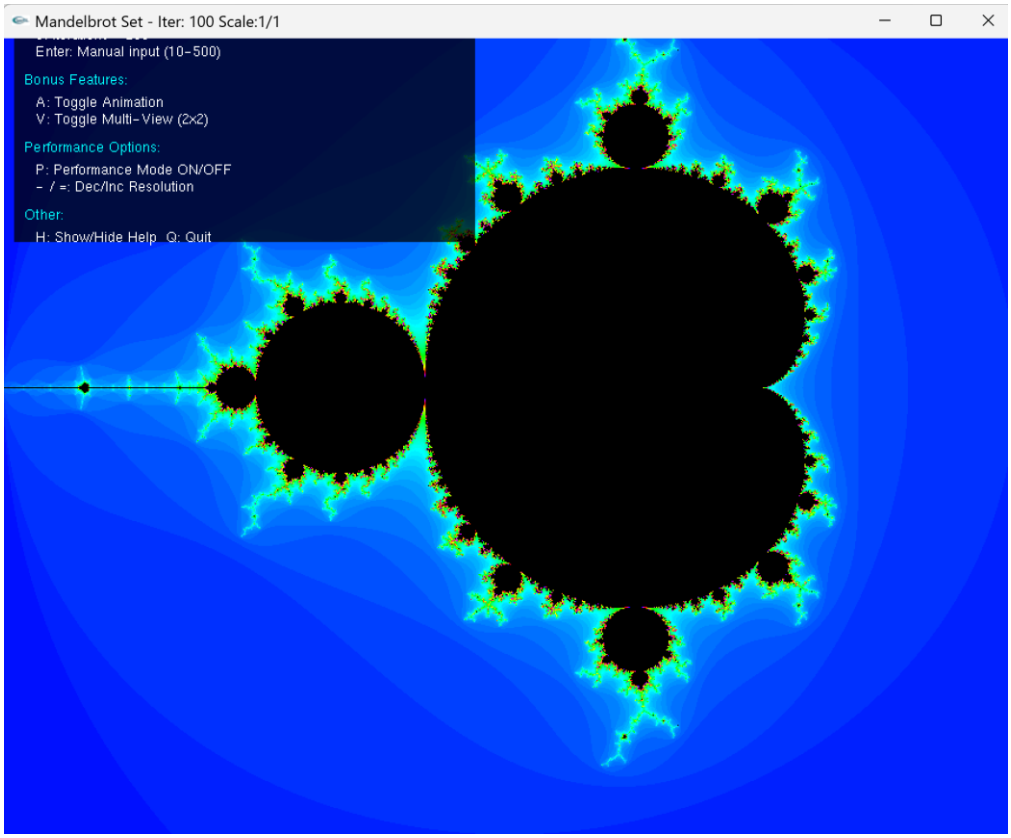


图 2-2 100 次迭代的分形效果（全分辨率）

按 “Enter” 键手动输入 “180”，分形细节进一步丰富（图 3）；

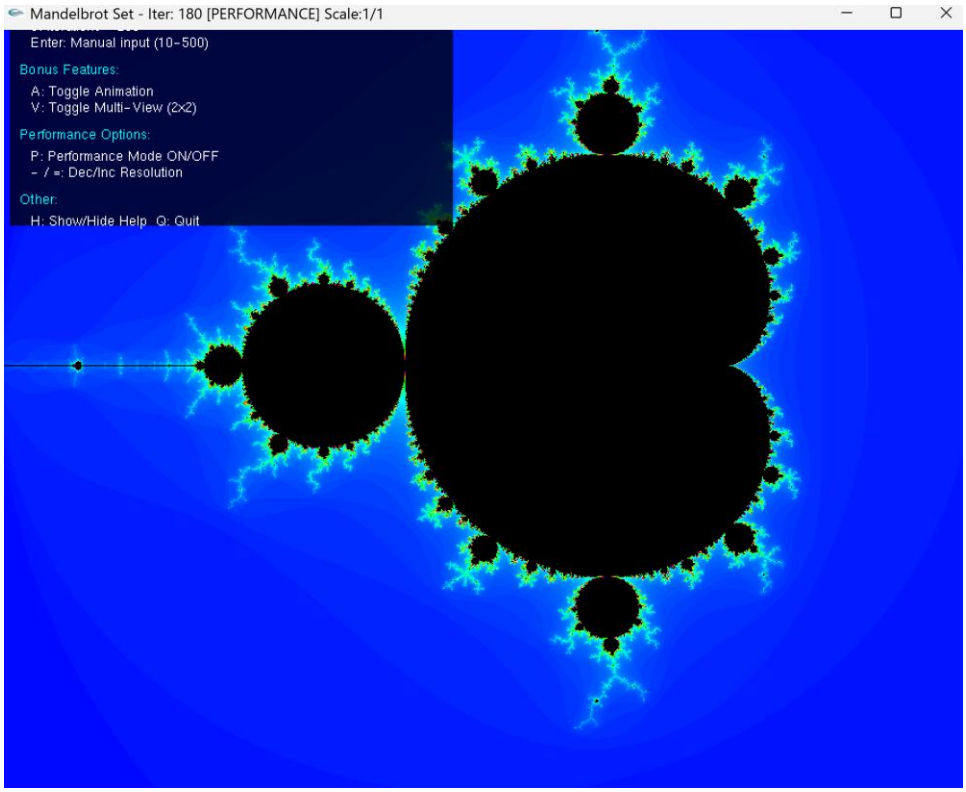


图 2-3 手动输入 180 次迭代的分形效果（全分辨率）

按 “P” 键开启性能模式，迭代次数限制为 50、渲染缩放为 2（半分辨率），分形以矩形块绘制，帧率提升，状态栏显示 “[PERF MODE] | Scale:1/2”（图 4）；

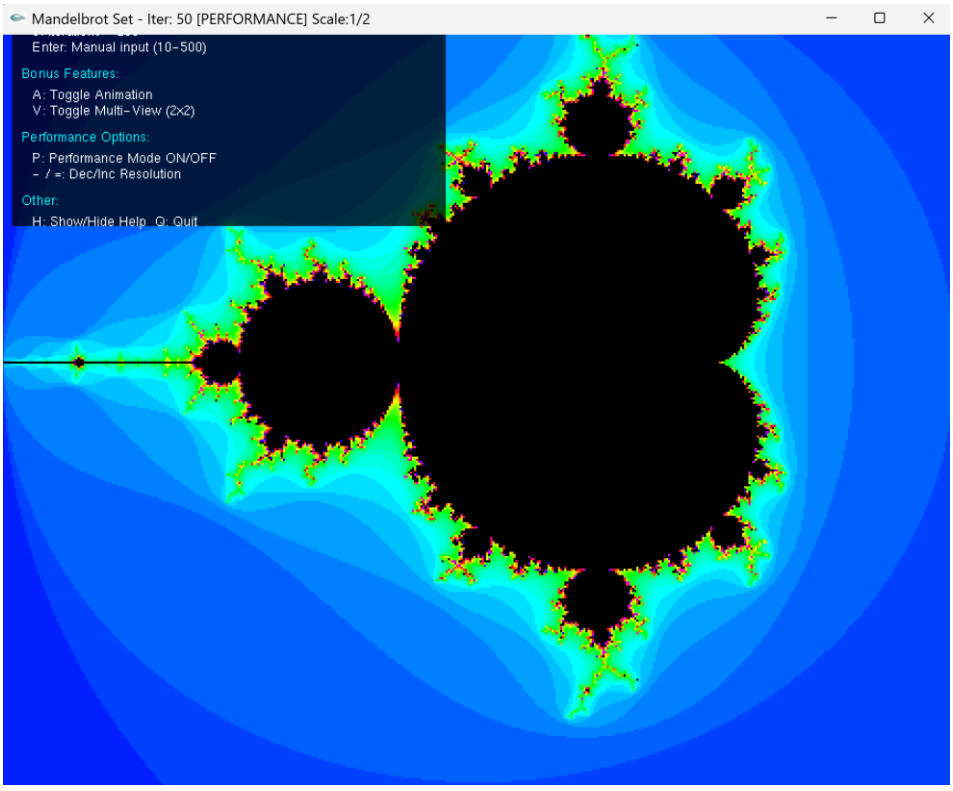


图 2-4 性能模式下的分形效果（50 次迭代，半分辨率）

步骤 3：动画与多视口对比

动画功能（A 键）：

代码逻辑：按 “A” 键切换动画开关，animationFunc 每 500ms 循环更新迭代次数，触发分形动态变化；

操作与效果：按 “A” 键，状态栏显示 “Anim: ON”，分形迭代次数循环变化（如 30→40→…→200→30），细节 “从粗到细再循环”（图 5）；

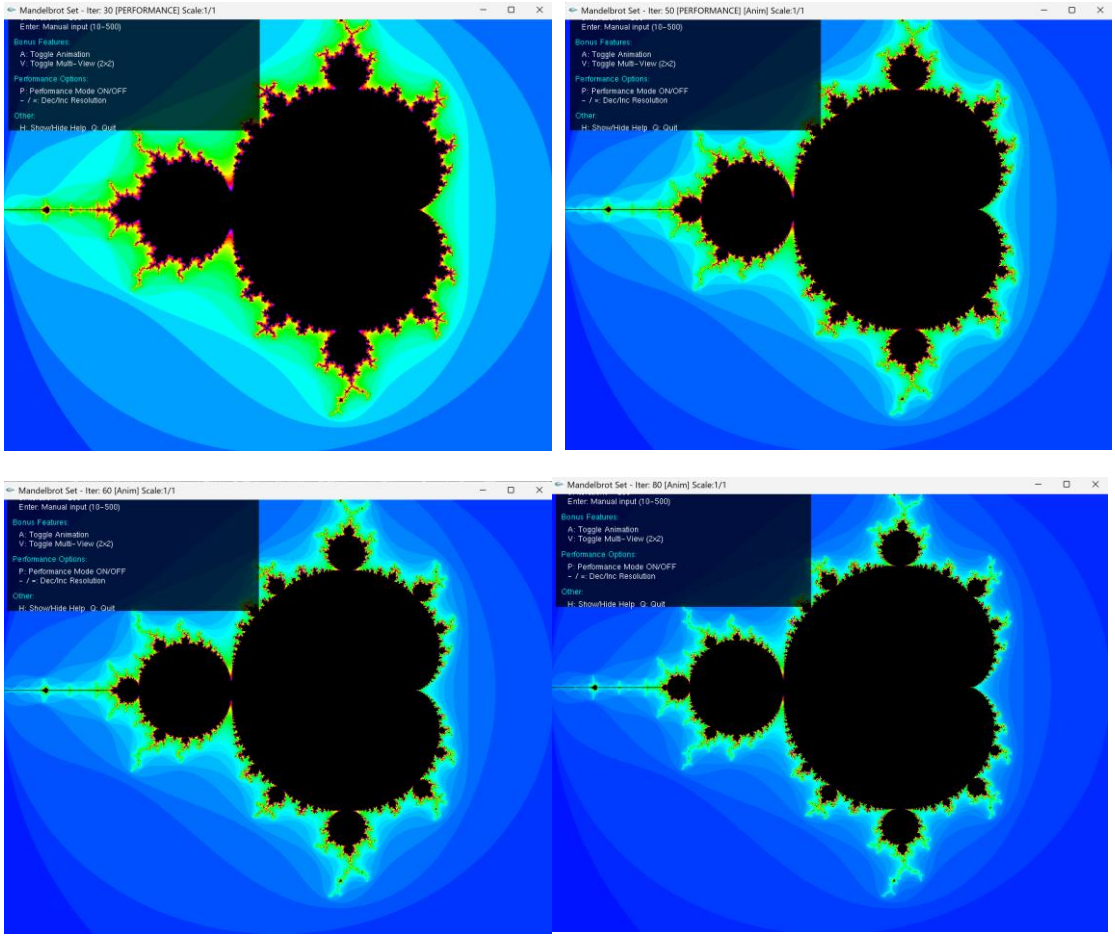


图 2-5 分形动画效果（某四帧，30-50-60-80）

多视口功能（V 键）：

代码逻辑：按 “V” 键切换 2×2 多视口，分别绘制 50/100/150/300 次迭代的分形，灰色分割线区分视口并标注迭代次数；

操作与效果：按 “V” 键，4 个视口直观对比迭代次数对细节的影响 ——50 次显主体轮廓，300 次呈丰富分支（图 6）；

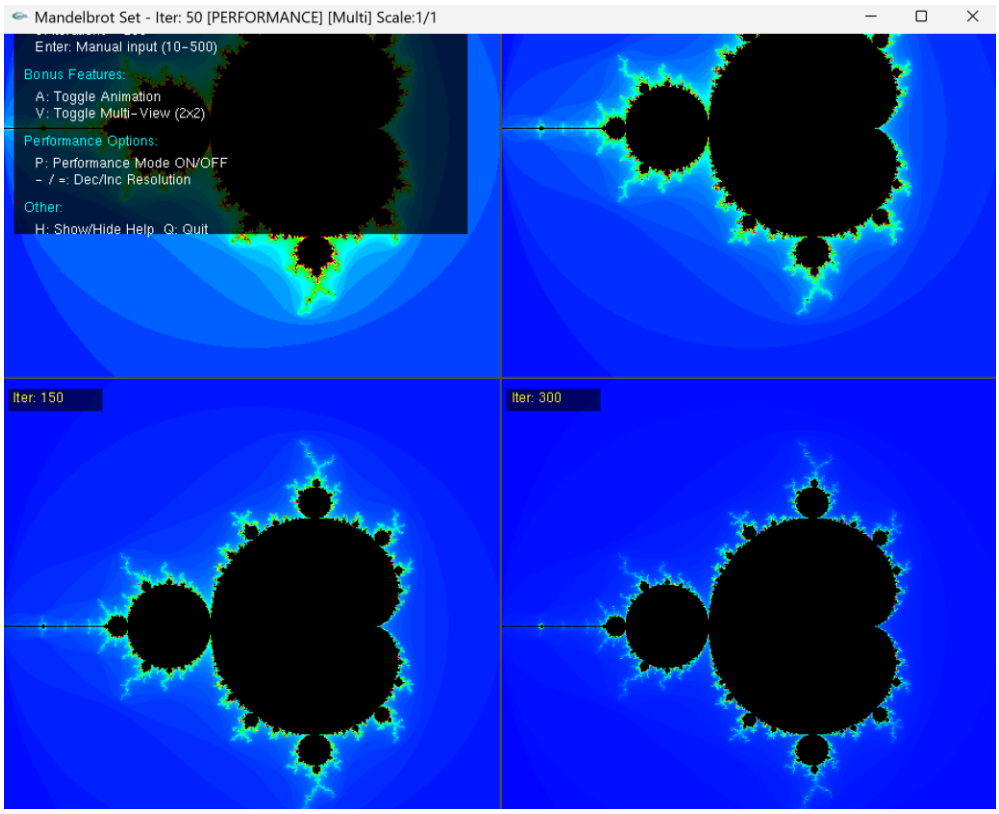


图 2-6 2×2 多视口对比效果（不同迭代次数分形）

Mandelbrot 分形模块最终结果展示及说明

最终界面（可实现功能组合）：开启 “多视口 + 性能模式” 并单独验证动画功能 ——

多视口 + 性能模式：按 “V” 键切换 2×2 多视口（分别显示 50/100/150/300 次迭代分形），按 “P” 键开启性能模式（渲染缩放 1/2，半分辨率绘制），状态栏显示 “Iter: 50 | Anim: OFF | View: Multi | [PERF MODE] | Scale:1/2”，窗口标题为 “Mandelbrot Set - Iter: 50 [PERFORMANCE] [Multi] Scale:1/2”；4 个视口分形对比清晰，半分辨率绘制下帧率稳定，无卡顿（图 7）；

单视口动画 + 性能模式：关闭多视口后按 “A” 键开启动画，迭代次数在 20-50 间循环（性能模式限制），状态栏显示 “Iter: 35 | Anim: ON | View: Single | [PERF MODE] | Scale:1/2”，分形动态变化流畅，支持按 “H” 键隐藏 / 显示帮助面板（图 8）。

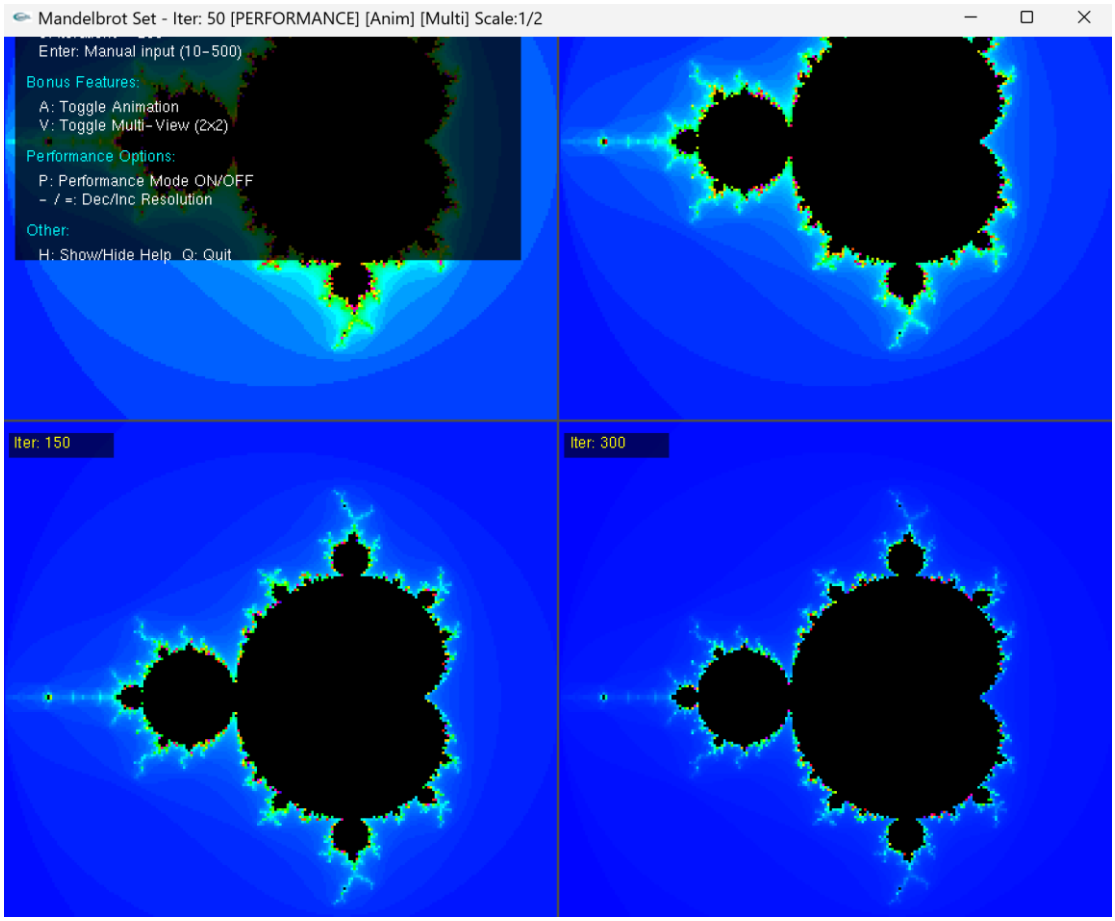


图 2-7 多视口 + 性能模式最终效果（4 个视口对比，半分辨率）

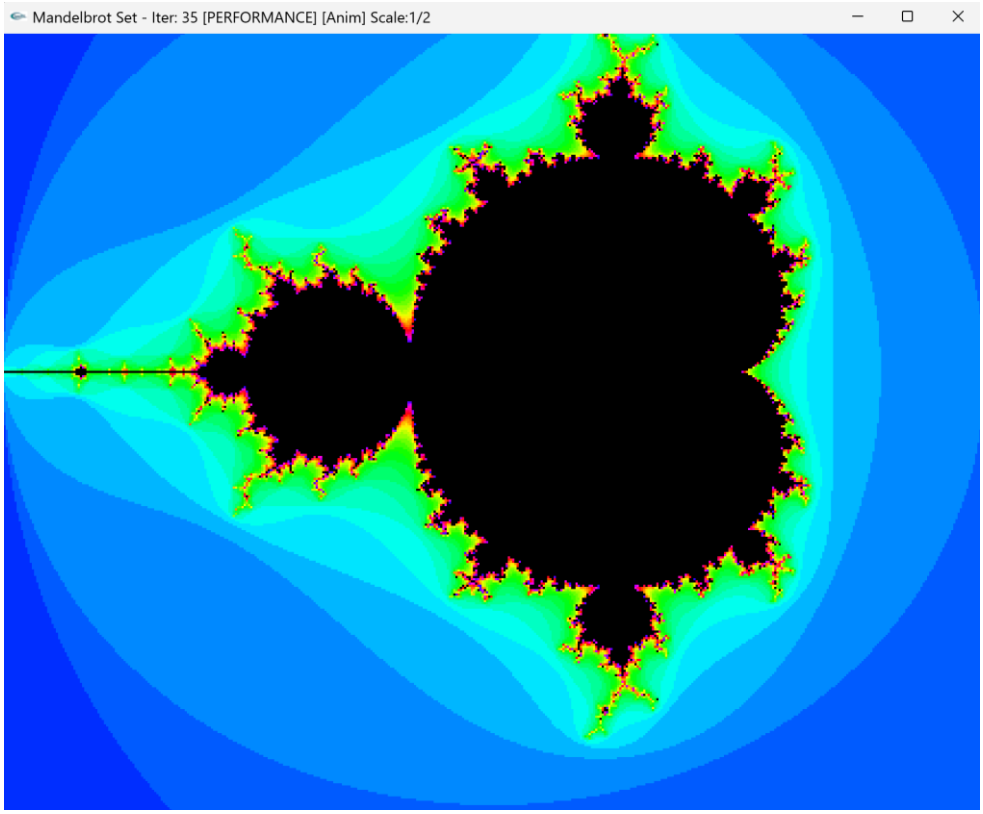


图 2-8 单视口动画 + 性能模式（迭代 35 次，半分辨率，隐藏窗口）

功能覆盖说明：在实现迭代次数控制、分形绘制和动画、多视口的基础上，新增性能优化特性（早期终止、降采样渲染、性能模式、分辨率调节），确保复杂场景下流畅运行。

五、总结及心得体会：

（一）实验过程中遇到的问题及解决办法

1.贝塞尔曲线控制点选中精度低：初始时鼠标点击控制点难以选中，原因是未设置合理距离判断阈值。解决办法:在 mouseFunc 中计算鼠标与控制点的欧氏距离，当距离<10.0f 时判定为选中，同时给选中点添加黑色描边，提升视觉辨识度；

2.Mandelbrot 分形颜色单调：初始用 RGB 固定颜色段，导致分形边缘颜色过渡生硬。解决办法：改用 HSV 颜色映射，按迭代次数 t=iter/maxIter 线性调整色相（0→360°），实现颜色平滑过渡；

3. 动画闪烁：贝塞尔曲线与分形动画均出现闪烁，原因是单缓冲绘制时 “清除 - 重绘” 过程可见。解决办法：在 glutInitDisplayMode 中启用双缓冲（GLUT_DOUBLE），通过 glutSwapBuffers 切换前后缓冲区，消除闪烁；

（二）心得体会

1.理论与实践结合：通过实现贝塞尔曲线的伯恩斯坦基函数，深刻理解 “参数曲线由控制点权重决定” 的原理；通过 Mandelbrot 集的复数迭代，直观感受到 “简单数学规则生成复杂图形” 的分形特性，打破了对 “复杂图形需复杂算法” 的认知；

2.交互设计的重要性：初始版本仅实现核心绘制功能，用户操作困难。后续添加状态栏、帮助面板、控制点选中反馈后，程序易用性大幅提升，意识到图形程序不仅要能画，更要好用；

3.OpenGL 渲染流程的掌握：从初始化、绘制、事件响应到缓冲区交换，逐步理清 OpenGL 的 “事件驱动 + 渲染循环” 逻辑。

六、对本实验过程及方法、手段的改进建议：

（一）对实验教学的建议

1.增加案例演示环节：实验前可展示贝塞尔曲线在 “矢量绘图软件钢笔工具”、分形在 “自然景观模拟” 中的实际应用，帮助学生理解实验的工程价值；

2.提供调试技巧指导：OpenGL 报错信息较少，建议补充 “坐标映射错误排查”

“双缓冲启用步骤”“鼠标坐标与窗口坐标转换” 等调试方法，减少学生因环境配置或逻辑细节卡壳的时间；

3.分组协作任务：可将实验拆分为 “曲线生成”“分形计算”“交互优化” 三个子任务，鼓励小组协作，培养分工与整合能力。

（二）对本项目的改进想法

1.功能扩展：

贝塞尔曲线模块：增加 “阶数切换” 功能（键盘快捷键切换 2 阶 / 3 阶曲线），支持 “曲线闭合”（首尾控制点连接）；

分形模块：增加鼠标滚轮缩放（调整复平面范围，查看分形局部细节）、拖拽平移功能，支持更多分形类型（如 Julia 集、Sierpinski 三角形）；

通用功能：添加“图像保存”（调用 glReadPixels 将当前帧保存为 BMP 格式）；

2.性能优化：

分形模块：采用 “帧缓存” 机制，缓存已计算的迭代结果，避免重复计算，提升动画帧率；

贝塞尔曲线模块：当控制点数量较多时，采用 “分段绘制” 减少单次循环计算量；

3.界面美化：

优化帮助面板样式（增加半透明背景），补充快捷键提示的图标；

贝塞尔曲线模块增加 “控制点颜色自定义” 功能，支持不同曲线用不同色系区分，提升视觉体验。

报告评分：

指导教师签字：